

Formation CDA

Angular – Formulaires & Authentification

Andrej RACIC - NUMERICA

Sommaire

- 1 — Reactive Forms et Angular Material forms
- 2 — Authentification complète et intercepteur HTTP

Reactive Forms et Angular Material forms

Objectifs

- Comprendre la différence entre Template-driven et Reactive Forms.
- Créer un formulaire avec FormGroup et FormControl.
- Simplifier la création avec FormBuilder.
- Ajouter des validateurs synchrones : required, minLength, email, pattern.
- Afficher les erreurs de validation dans le template.
- Utiliser les composants Material pour les champs de formulaire.

1. Reactive Forms vs Template-driven Forms

Angular propose les formulaires pilotés par le template et les Reactive Forms. Ce cours utilise les Reactive Forms, stables et adaptées aux formulaires complexes, car leur modèle et leur validation sont définis en TypeScript. Angular 21 a introduit les Signal Forms comme API expérimentale ; elles sont stables à partir d'Angular 22. Ce cours conserve néanmoins les Reactive Forms.

	Template-driven	Reactive Forms
Logique définie dans	Le template HTML	La classe TypeScript
Validation	Attributs HTML (required...)	Validators TypeScript
Testabilité	Difficile	Facile — pure logique TS
Formulaires complexes	Limité	Idéal
Utilisation dans ce cours	Non	Oui

2. FormControl et FormGroup

Un FormControl représente un seul champ de formulaire. Il contient la valeur actuelle, l'état de validation et les erreurs. Un FormGroup est un groupe de FormControl qui forme un formulaire complet — il valide tous ses contrôles et expose leur valeur sous forme d'objet.

Analogie : FormGroup comme un questionnaire

Un questionnaire (FormGroup) contient plusieurs questions (FormControl).
Chaque question a sa propre réponse (value) et ses propres règles (validators).
Le questionnaire est valide seulement quand TOUTES les questions sont valides.

Configurer ReactiveFormsModule

```
// Dans le composant standalone – importer ReactiveFormsModule
import { ReactiveFormsModule } from '@angular/forms';

@Component({
  imports:      [ReactiveFormsModule], // indispensable pour
  formGroup/formControl
  template:    '...',
})
export class MonFormulaire { }
```

Créer un formulaire avec FormGroup et FormControl

```
import { Component }                from '@angular/core';
import { FormGroup, FormControl, Validators } from '@angular/forms';
import { ReactiveFormsModule }      from '@angular/forms';

@Component({
  imports:      [ReactiveFormsModule],
  templateUrl: './login.html',
})
export class Login {
  // FormGroup = le formulaire complet
  // FormControl = un champ individuel avec sa valeur initiale et ses
  // validateurs
  loginForm = new FormGroup({
    email:      new FormControl('', [Validators.required,
    Validators.email]),
    motDePasse: new FormControl('', [Validators.required,
    Validators.minLength(6)]),
  });

  // Accès rapide aux contrôles individuels (pour le template)
  get email()      { return this.loginForm.get('email')!; }
  get motDePasse() { return this.loginForm.get('motDePasse')!; }

  onSubmit(): void {
    if (this.loginForm.invalid) return; // bloquer si invalide

    // loginForm.value = { email: '...', motDePasse: '...' }
  }
}
```

```

    console.log('Formulaire soumis :', this.loginForm.value);
  }
}

```

Template du formulaire

```

<!-- login.html -->
<!-- formGroup connecte le <form> HTML au FormGroup TypeScript -->
<form [formGroup]='loginForm' (ngSubmit)='onSubmit()'>

  <div>
    <label>Email</label>
    <!-- formControlName connecte l'input au FormControl 'email' -->
    <input formControlName='email' type='email' placeholder='Votre
email'>

    <!-- Afficher les erreurs seulement si le champ a été touché -->
    @if (email.invalid && email.touched) {
      @if (email.errors?.['required']) {
        <span class='erreur'>L'email est requis</span>
      }
      @if (email.errors?.['email']) {
        <span class='erreur'>Format email invalide</span>
      }
    }
  </div>

  <div>
    <label>Mot de passe</label>
    <input formControlName='motDePasse' type='password'>
    @if (motDePasse.invalid && motDePasse.touched) {
      @if (motDePasse.errors?.['required']) {
        <span class='erreur'>Le mot de passe est requis</span>
      }
      @if (motDePasse.errors?.['minlength']) {
        <span class='erreur'>6 caractères minimum</span>
      }
    }
  </div>

  <!-- [disabled] désactive le bouton si le formulaire est invalide -->
  <button type='submit' [disabled]='loginForm.invalid'>
    Se connecter
  </button>
</form>

```

touched vs dirty vs valid

Un FormControl a plusieurs états booléens :

- **touched** : l'utilisateur a cliqué dans le champ puis l'a quitté (blur). On affiche les erreurs seulement si touched pour ne pas stresser l'utilisateur avant qu'il ait commencé à remplir.
- **dirty** : l'utilisateur a modifié la valeur du champ.
- **valid / invalid** : tous les validateurs passent ou non.
- **pristine** : le champ n'a pas encore été modifié (opposé de dirty).

3. FormBuilder — simplifier la création

FormBuilder est un service Angular qui fournit des méthodes raccourcies pour créer des FormGroup et FormControl. Le résultat est identique à new FormGroup / new FormControl, mais la syntaxe est plus concise et lisible.

```
import { Component, inject } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';
import { ReactiveFormsModule } from '@angular/forms';

@Component({
  imports: [ReactiveFormsModule],
  templateUrl: './register.html',
})
export class Register {
  private fb = inject(FormBuilder);

  // fb.group() est équivalent à new FormGroup()
  // Syntaxe : [valeurInitiale, validateurs] ou [valeurInitiale]
  registerForm = this.fb.group({
    nom: ['', [Validators.required, Validators.minLength(2)]],
    email: ['', [Validators.required, Validators.email]],
    motDePasse: ['', [Validators.required, Validators.minLength(6)]],
    confirmation: ['', [Validators.required]],
    genre: [''],
  });

  get nom() { return this.registerForm.get('nom')!; }
  get email() { return this.registerForm.get('email')!; }
  get motDePasse() { return this.registerForm.get('motDePasse')!; }
  get confirmation() { return this.registerForm.get('confirmation')!; }

  onSubmit(): void {
    if (this.registerForm.invalid) return;
    const { nom, email, motDePasse } = this.registerForm.value;
```

```

    console.log('Inscription :', { nom, email, motDePasse });
  }
}

```

4. Les validateurs intégrés

Angular fournit des validateurs prêts à l'emploi dans la classe Validators. Chaque validateur retourne null si la valeur est valide, ou un objet d'erreur si elle est invalide. On peut combiner plusieurs validateurs dans un tableau.

Validateur	Utilisation	Clé d'erreur
Validators.required	Champ obligatoire	errors['required']
Validators.minLength(n)	Longueur minimale n caractères	errors['minlength']
Validators.maxLength(n)	Longueur maximale n caractères	errors['maxlength']
Validators.email	Format email valide	errors['email']
Validators.min(n)	Valeur numérique >= n	errors['min']
Validators.max(n)	Valeur numérique <= n	errors['max']
Validators.pattern(regex)	Correspond à l'expression régulière	errors['pattern']

Validateur personnalisé

```

import { AbstractControl, ValidationErrors } from '@angular/forms';

// Un validateur personnalisé est une fonction qui prend un
// AbstractControl
// et retourne null (valide) ou un objet d'erreur (invalide)
function motsDePasseIdentiques(control: AbstractControl):
ValidationErrors | null {
  const mdp = control.get('motDePasse')?.value;
  const confirm = control.get('confirmation')?.value;

  if (mdp && confirm && mdp !== confirm) {
    // Retourner un objet d'erreur - la clé est le nom de l'erreur
    return { motsDePasseDifférents: true };
  }
  return null; // valide
}

// Appliquer au FormGroup (pas à un FormControl individuel)

```

```

registerForm = this.fb.group({
  motDePasse: ['', [Validators.required, Validators.minLength(6)]],
  confirmation: ['', Validators.required],
},
{ validators: motsDePasseIdentiques } // validateur au niveau du
groupe
);

// Dans le template
@if (registerForm.errors?.['motsDePasseDifférents'] &&
confirmation.touched) {
  <span class='erreur'>Les mots de passe ne correspondent pas</span>
}

```

5. Angular Material — champs de formulaire

Angular Material fournit des composants de formulaire qui s'intègrent parfaitement avec les Reactive Forms. `MatFormField` est le conteneur qui apporte le style Material (label flottant, ligne de soulignement, messages d'erreur). Il s'utilise avec `matInput`, `MatSelect`, `MatCheckbox`...

```

// Imports nécessaires
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule }      from '@angular/material/input';
import { MatSelectModule }     from '@angular/material/select';
import { MatButtonModule }     from '@angular/material/button';
import { MatIconModule }       from '@angular/material/icon';
// imports: [ReactiveFormsModule, MatFormFieldModule, MatInputModule,
//           MatSelectModule, MatButtonModule, MatIconModule]

<!-- Formulaire Material complet -->
<form [formGroup]='registerForm' (ngSubmit)='onSubmit()'>

  <!-- mat-form-field = conteneur Material pour un champ -->
  <mat-form-field appearance='outline'>
    <!-- mat-label = label flottant -->
    <mat-label>Nom complet</mat-label>
    <!-- matInput = applique le style Material à l'input natif -->
    <input matInput formControlName='nom' placeholder='Alice Dupont'>
    <mat-icon matSuffix>person</mat-icon>
    <!-- mat-error = affiché si le contrôle est invalide et touché, ou
après soumission du formulaire -->
    <mat-error>
      @if (nom.errors?.['required']) { Le nom est requis. }
      @if (nom.errors?.['minlength']) { 2 caractères minimum. }
    </mat-error>
  </mat-form-field>

```

```

<mat-form-field appearance='outline'>
  <mat-label>Email</mat-label>
  <input matInput formControlName='email' type='email'>
  <mat-icon matSuffix>email</mat-icon>
  <mat-error>
    @if (email.errors?.['required']) { L'email est requis. }
    @if (email.errors?.['email']) { Format email invalide. }
  </mat-error>
</mat-form-field>

<mat-form-field appearance='outline'>
  <mat-label>Mot de passe</mat-label>
  <input matInput formControlName='motDePasse' [type]='afficherMdp ?
"text" : "password"'>
  <!-- Bouton pour afficher/masquer le mot de passe -->
  <button mat-icon-button matSuffix type='button'
    aria-label='Afficher ou masquer le mot de passe'
    (click)='afficherMdp = !afficherMdp'>
    <mat-icon>{{ afficherMdp ? 'visibility_off' : 'visibility'
  }}</mat-icon>
  </button>
  <mat-error>
    @if (motDePasse.errors?.['required']) { Le mot de passe est
requis. }
    @if (motDePasse.errors?.['minlength']) { 6 caractères minimum. }
  </mat-error>
</mat-form-field>

<!-- mat-select = liste déroulante Material -->
<mat-form-field appearance='outline'>
  <mat-label>Genre préféré</mat-label>
  <mat-select formControlName='genre'>
    <mat-option value='roman'>Roman</mat-option>
    <mat-option value='essai'>Essai</mat-option>
    <mat-option value='bd'>Bande dessinée</mat-option>
  </mat-select>
</mat-form-field>

  <button mat-flat-button type='submit'
[disabled]='registerForm.invalid'>
    S'inscrire
  </button>
</form>

```

mat-error vs message d'erreur custom

mat-error est affiché par Angular Material lorsque le FormControl est invalide et que l'utilisateur a touché le champ, ou lorsque le formulaire parent a été soumis. Il n'est donc généralement pas nécessaire d'ajouter vous-même la condition @if (control.touched). Ce comportement peut être personnalisé avec ErrorStateMatcher.

Exercices

Continuez sur le projet bibliotheque-front. Générez les composants nécessaires avec ng g c.

Exercice 1 — Formulaire de connexion avec FormGroup

Générez ng g c pages/login. Dans Login, créez un loginForm avec FormGroup contenant email (required, email) et motDePasse (required, minLength(6)). Ajoutez les getters email et motDePasse. Implémentez onSubmit() qui vérifie loginForm.invalid avant d'afficher la valeur en console. Dans le template, créez un <form [formGroup]> avec deux inputs connectés via FormControlName, les messages d'erreur conditionnels (avec touched) et un bouton submit désactivé si invalide.

Exercice 2 — Formulaire d'inscription avec FormBuilder et validateurs

Générez ng g c pages/register. Utilisez FormBuilder (injecté avec inject()) pour créer un registerForm avec : nom (required, minLength(2)), email (required, email), motDePasse (required, minLength(6)), confirmation (required). Dans onSubmit(), affichez les valeurs sans le champ confirmation. Ajoutez les getters pour chaque contrôle et les messages d'erreur correspondants dans le template.

Exercice 3 — Validateur personnalisé — mots de passe identiques

Ajoutez dans Register un validateur personnalisé motsDePasseIdentiques qui compare motDePasse et confirmation au niveau du FormGroup. Appliquez-le avec { validators: motsDePasseIdentiques } dans fb.group(). Dans le template, affichez le message 'Les mots de passe ne correspondent pas' si l'erreur motsDePasseDifférents est présente sur le registerForm et que confirmation.touched est true.

Exercice 4 — Refactoriser avec Angular Material form fields

Refactorisez Login et Register pour utiliser Angular Material : remplacez les inputs natifs par mat-form-field avec matInput et mat-label, ajoutez des icônes Material adaptées (email, lock, visibility...) en matSuffix, utilisez mat-error pour les messages (Material gère l'état d'erreur : champ touché ou formulaire soumis), remplacez le bouton par mat-flat-button. Ajoutez sur le champ mot de passe un bouton matSuffix pour afficher/masquer la valeur.

Exercice 5 — Formulaire de création de livre

Créez ng g c composants/formulaire-livre. Ce formulaire permet à un admin de

créer un livre avec : titre (required), auteur (required), isbn (required, pattern /^[0-9-]{10,17}\$/), annee (min(1000), max(2100)), genre (mat-select avec les 5 genres). Dans onSubmit(), appelez livreService.creerLivre() et émettez un @Output() livreCree = new EventEmitter<Livre>() avec le livre créé. Affichez un MatSnackBar de confirmation.

Exercice 6 — Réinitialiser et pré-remplir un formulaire

Dans FormulaireLivre, ajoutez une méthode reinitialiser() qui appelle this.livreForm.reset() pour vider tous les champs. Ajoutez aussi une méthode preremplir(livre: Partial<Livre>) qui appelle this.livreForm.patchValue(livre) pour pré-remplir le formulaire avec les données d'un livre existant (mode édition). Testez les deux fonctions avec des boutons dédiés dans le template.

Authentification complète et intercepteur HTTP

Objectifs

- Connecter le formulaire de login à l'API Node.js.
- Stocker le token JWT et l'utilisateur dans localStorage après le login.
- Créer un AuthService qui centralise l'état d'authentification.
- Implémenter un intercepteur HTTP pour injecter automatiquement le token.
- Mettre à jour le Guard pour utiliser AuthService.
- Gérer la déconnexion et la redirection.

1. AuthService — centraliser l'authentification

L'AuthService est le service central de l'authentification. Il gère le stockage du token et de l'utilisateur dans localStorage, expose des méthodes pour se connecter/déconnecter, et fournit des observables pour connaître l'état de connexion en temps réel.

```
// src/app/services/auth.service.ts
import { Injectable, computed, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { Observable, tap } from 'rxjs';

export interface User {
  _id: string;
  nom: string;
  email: string;
  role: 'user' | 'admin';
}

export interface ApiResponse<T> { success: boolean; data: T; }
export interface AuthData { token: string; user: User; }
export interface RegisterPayload { nom: string; email: string;
mot_de_passe: string; }

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
```

```

private readonly router = inject(Router);
private readonly baseUrl = 'http://localhost:3000/auth';

// Signal : valeur réactive qui notifie les composants quand elle
change
// On les verra en détail dans le cours suivant – ici on les utilise
simplement
private readonly _user = signal<User |
null>(this.lireUserDuStorage());

// computed : valeur dérivée du signal _user
readonly user = this._user.asReadonly();
readonly estConnecte = computed(() => this._user() !== null);
readonly estAdmin = computed(() => this._user()?.role === 'admin');

// Lire l'utilisateur stocké au démarrage de l'app
private lireUserDuStorage(): User | null {
  const str = localStorage.getItem('user');
  if (!str) return null;
  try { return JSON.parse(str) as User; }
  catch { return null; }
}

// Se connecter – appelle POST /auth/login
login(email: string, motDePasse: string):
Observable<ApiResponse<AuthData>> {
  return this.http.post<ApiResponse<AuthData>>(`
    ${this.baseUrl}/login`,
    { email, mot_de_passe: motDePasse },
  ).pipe(
    tap(reponse => {
      // Stocker le token et l'utilisateur
      localStorage.setItem('token', reponse.data.token);
      localStorage.setItem('user', JSON.stringify(reponse.data.user));
      // Mettre à jour le signal
      this._user.set(reponse.data.user);
    }),
  );
}

// S'inscrire – appelle POST /auth/register
register(nom: string, email: string, motDePasse: string):
Observable<ApiResponse<User>> {
  const payload: RegisterPayload = { nom, email, mot_de_passe:
motDePasse };
  return this.http.post<ApiResponse<User>>(`
    ${this.baseUrl}/register`,
    payload);
}

```

```
// Se déconnecter — nettoyer localStorage et rediriger
logout(): void {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  this._user.set(null);
  this.router.navigate(['/login']);
}

// Lire le token pour les requêtes HTTP
getToken(): string | null {
  return localStorage.getItem('token');
}
}
```

signal() et computed() — aperçu avant le cours suivant

On utilise ici deux concepts que l'on verra dans le prochain cours : `signal()` crée une valeur réactive (le composant navbar se met à jour automatiquement quand `_user` change) et `computed()` crée une valeur dérivée (`estConnecte` est recalculé automatiquement quand `_user` change). Pour l'instant, utilisez-les comme indiqué — vous comprendrez leur mécanique complète dans le cours suivant.

2. Connecter le formulaire de login à l'API

Maintenant qu'on a l'`AuthService`, on connecte le formulaire de login pour appeler l'API Node.js et stocker le token reçu.

```
// login.ts — version complète
import { Component, inject } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';
import { ReactiveFormsModule } from '@angular/forms';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatSnackBar } from '@angular/material/snack-bar';
import { AuthService } from '../services/auth.service';

@Component({
  imports: [ReactiveFormsModule, MatFormFieldModule, MatInputModule,
```

```

        MatButtonModule, MatIconModule, RouterLink],
        templateUrl: './login.html',
    })
    export class Login {
        private fb          = inject(FormBuilder);
        private authService = inject(AuthService);
        private router       = inject(Router);
        private route        = inject(ActivatedRoute);
        private snackBar     = inject(MatSnackBar);

        afficherMdp = false;
        chargement  = false;
        erreur      = '';

        loginForm = this.fb.group({
            email:      ['', [Validators.required, Validators.email]],
            motDePasse: ['', [Validators.required, Validators.minLength(6)]],
        });

        get email()      { return this.loginForm.get('email')!; }
        get motDePasse() { return this.loginForm.get('motDePasse')!; }

        onSubmit(): void {
            if (this.loginForm.invalid) return;
            this.chargement = true;
            this.erreur     = '';

            const { email, motDePasse } = this.loginForm.value;

            this.authService.login(email!, motDePasse!).subscribe({
                next: () => {
                    this.chargement = false;
                    this.snackBar.open('Connexion réussie !', 'Fermer', { duration:
2000 });
                    const returnUrl =
this.route.snapshot.queryParamMap.get('returnUrl') ?? '/';
                    this.router.navigateByUrl(returnUrl);
                },
                error: (err) => {
                    this.chargement = false;
                    this.erreur = err.error?.message || 'Email ou mot de passe
incorrect';
                },
            });
        }
    }
}

```

3. L'intercepteur HTTP — injecter le token automatiquement

Sans intercepteur, il faut ajouter manuellement le header Authorization dans chaque méthode de service qui appelle une route protégée. Un intercepteur HTTP est une fonction qui s'exécute automatiquement avant chaque requête HTTP — il peut modifier la requête (ajouter des headers) ou la réponse (intercepter les erreurs 401).

Analogie : l'intercepteur comme un douanier automatique

Sans intercepteur, chaque développeur doit penser à montrer son passeport (token) à chaque frontière (route protégée). Avec un intercepteur, un douanier automatique (l'intercepteur) vérifie le passeport de TOUS les voyageurs (requêtes HTTP) avant qu'ils passent — vous n'avez plus à y penser.

Créer l'intercepteur

```
// src/app/interceptors/auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

// HttpInterceptorFn = type d'un intercepteur fonctionnel (Angular 15+)
// req = la requête originale (immuable)
// next = la fonction pour passer au suivant dans la chaîne
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.getToken();

  // Si pas de token, laisser passer la requête telle quelle
  // (routes publiques : login, register, GET /livres...)
  if (!token) return next(req);

  // Cloner la requête en ajoutant le header Authorization
  // On clone car HttpRequest est immuable
  const reqAvecToken = req.clone({
    setHeaders: {
      Authorization: `Bearer ${token}`,
    },
  });

  return next(reqAvecToken);
};
```

Enregistrer l'intercepteur dans app.config.ts

```
// src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withInterceptors } from
 '@angular/common/http';
import { provideRouter } from '@angular/router';
import { authInterceptor } from '../interceptors/auth.interceptor';
import { routes } from '../app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    // HttpClient existe par défaut en Angular 21+, mais
    provideHttpClient
    // est nécessaire ici pour enregistrer les intercepteurs.
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
};

// Maintenant TOUTES les requêtes HTTP auront automatiquement
// le header Authorization: Bearer <token> si un token est présent
// Plus besoin d'HttpHeaders dans les services !
```

Nettoyer les services — supprimer les headers manuels

```
// livre.service.ts — AVANT l'intercepteur
supprimerLivre(id: string): Observable<ApiResponse<null>> {
  const token = localStorage.getItem('token') || '';
  const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
  return this.http.delete<ApiResponse<null>>(`${this.baseUrl}/${id}`, {
    headers }); // lourd
}

// livre.service.ts — APRÈS l'intercepteur
supprimerLivre(id: string): Observable<ApiResponse<null>> {
  return this.http.delete<ApiResponse<null>>(`${this.baseUrl}/${id}`);
  // propre !
  // L'intercepteur ajoute automatiquement le token
}
```

4. Gérer les erreurs 401 dans l'intercepteur

L'intercepteur peut aussi intercepter les réponses HTTP pour gérer les erreurs globalement. Une erreur 401 (token expiré ou invalide) doit déconnecter l'utilisateur et le rediriger vers /login automatiquement.

```
// auth.interceptor.ts – version complète avec gestion 401
import { HttpResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../services/auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getToken();
  const endpointAuth = req.url.includes('/auth/login') ||
    req.url.includes('/auth/register');

  const requete = token && !endpointAuth
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(requete).pipe(
    catchError((err: HttpResponse) => {
      // Un 401 sur login signifie simplement identifiants invalides.
      if (err.status === 401 && token && !endpointAuth) {
        auth.logout();
      }
      return throwError(() => err);
    })
  );
};
```

5. Navbar dynamique avec l'état d'authentification

Maintenant qu'AuthService expose le signal user et le computed estConnecté, on peut rendre la navbar dynamique : afficher le nom de l'utilisateur connecté, un bouton de déconnexion, et masquer le lien de login si déjà connecté.

```
// app.ts
import { Component, inject } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { AuthService } from '../services/auth.service';
```

```

@Component({
  imports: [RouterOutlet, RouterLink, MatToolbarModule,
    MatButtonModule, MatIconModule],
  templateUrl: './app.html',
})
export class App {
  authService = inject(AuthService);
  // authService.user() = signal -> valeur actuelle de l'utilisateur
  // authService.estConnecte() = computed -> true si user !== null
}
<!-- app.html -->
<mat-toolbar>
  <span>Bibliothèque CDA</span>
  <span class='spacer'></span>

  <!-- Afficher selon l'état de connexion -->
  @if (authService.estConnecte()) {
    <!-- Utilisateur connecté -->
    <span>{{ authService.user()?.nom }}</span>
    @if (authService.estAdmin()) {
      <a mat-button routerLink='/admin'>Admin</a>
    }
    <button mat-icon-button aria-label='Se déconnecter'
      (click)='authService.logout()'>
      <mat-icon>logout</mat-icon>
    </button>
  } @else {
    <!-- Visiteur non connecté -->
    <a mat-button routerLink='/login'>Se connecter</a>
  }
</mat-toolbar>

<router-outlet></router-outlet>

```

6. Mettre à jour le Guard pour utiliser AuthService

Le Guard du cours précédent lisait directement localStorage. Maintenant qu'on a AuthService, le Guard peut l'utiliser directement — c'est plus propre et cohérent avec le reste de l'application.

```

// src/app/guards/auth.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

```

```

export const authGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  return auth.estConnecte()
    ? true
    : router.createUrlTree(['/login'], {
        queryParams: { returnUrl: state.url },
      });
};

// Le Guard protège la navigation seulement.
// L'API doit toujours vérifier le token et les autorisations côté
// serveur.

// src/app/guards/admin.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

export const adminGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (!auth.estConnecte()) {
    return router.createUrlTree(['/login'], {
      queryParams: { returnUrl: state.url },
    });
  }

  if (authService.estAdmin()) return true;

  return router.createUrlTree(['/']);
};

```

Flux complet d'authentification

Voici le flux complet une fois tout en place :

- L'utilisateur remplit le formulaire de login et clique 'Se connecter'.
- Login appelle `authService.login()` -> POST `/auth/login` vers l'API Node.js.
- L'API retourne `{ success: true, data: { token, user } }`.
- `authService` stocke token et user dans `localStorage`, met à jour le signal.
- Login navigue vers `returnUrl` lorsqu'il est présent, sinon vers `/`.
- La navbar se met à jour automatiquement (signal réactif) : affiche le nom.
- Les prochaines requêtes HTTP incluent automatiquement le token (intercepteur).
- Les Guards utilisent `authService.estConnecte()` pour autoriser la navigation.

Exercices

Continuez sur le projet bibliotheque-front avec l'API Node.js lancée sur localhost:3000.

Exercice 1 — Créer AuthService avec login, register et logout

Générez ng g s services/auth. Implémentez AuthService avec : un signal `_user` initialisé depuis `localStorage` (`JSON.parse`), les computed `estConnecte` et `estAdmin`, les méthodes `login()` (POST `/auth/login`, tap pour stocker `token+user+mettre à jour signal`), `register()` (POST `/auth/register`) et `logout()` (`removeItem` token et user, set null, navigate `/login`). Exportez `getToken()` pour l'intercepteur.

Exercice 2 — Connecter le formulaire de login à l'API

Dans `Login`, injectez `AuthService`, `Router` et `ActivatedRoute`. Dans `onSubmit()`, appelez `authService.login()` avec les valeurs du formulaire. En cas de succès, affichez un `MatSnackBar` 'Connexion réussie' et naviguez vers le query param `returnUrl` s'il existe, sinon vers `/`. En cas d'erreur, affichez le message d'erreur de l'API (`err.error?.message`) dans une propriété `erreur` affichée dans le template. Testez avec des identifiants valides et invalides.

Exercice 3 — Créer et enregistrer l'intercepteur HTTP

Créez `src/app/interceptors/auth.interceptor.ts` avec la fonction `authInterceptor` de type `HttpInterceptorFn`. Elle doit : récupérer le token via `authService.getToken()`, cloner la requête avec `setHeaders` si token présent, intercepter les erreurs 401 avec `catchError` pour appeler `authService.logout()` seulement lorsqu'un token était présent et que la requête ne concernait pas `/auth/login` ou `/auth/register`. Enregistrez l'intercepteur dans `app.config.ts` avec `provideHttpClient(withInterceptors([authInterceptor]))`.

Exercice 4 — Nettoyer les services — supprimer les headers manuels

Dans `LivreService` et `EmpruntService`, supprimez toutes les références à `HttpHeaders` et `localStorage` — l'intercepteur s'en charge maintenant. Les appels HTTP ne doivent plus contenir d'options headers liées à l'authentification ; conservez les éventuels `pipe()`, `map()` et `catchError()` nécessaires au traitement des réponses. Vérifiez que les routes protégées fonctionnent toujours après avoir retiré les headers manuels.

Exercice 5 — Navbar dynamique avec l'état d'authentification

Dans `App`, injectez `AuthService`. Dans le template de la toolbar, utilisez `@if (authService.estConnecte())` pour afficher : si connecté -> nom de l'utilisateur (`authService.user()?.nom`), lien Admin si `estAdmin()`, bouton logout (`mat-icon-button` avec icône 'logout') qui appelle `authService.logout()`. Si non connecté -> un lien 'Se connecter' vers `/login`. Testez le cycle complet login -> navbar mise à jour -> logout -> retour au lien login.

Exercice 6 — Mettre à jour les Guards pour utiliser AuthService

Mettez à jour `auth.guard.ts` et `admin.guard.ts` pour utiliser `authService.estConnecte()` et `authService.estAdmin()` au lieu de lire `localStorage`

directement. Testez les scénarios : non connecté -> /emprunts redirige vers /login, connecté user -> /admin redirige vers /, connecté admin -> /admin accessible. Vérifiez que logout puis navigation vers /emprunts redirige bien vers /login.